

ATestaPonto

Documentação da API

Esta documentação descreve os contratos de API do sistema ATestaPonto, um sistema de controle de presença de funcionários em ambiente escolar.

Identificação do projeto

Projeto	ATestaPonto
Finalidade	Sistema de controle de presença de funcionários em ambiente escolar
Backend	Java Spring boot
Banco de dados	MySQL
Autenticação (admin)	Gov.br OAuth 2.0 + sessão server-side (Redis opcional)
Autenticação (funcionário)	JWT (JSON Web Token) com validação no middleware
Prefixos da API	/api/admin /ponto /auth/govbr

Sumário

Introdução	5
1. Visão geral	5
Arquitetura	5
2. Autenticação e autorização	5
2.1 Autenticação de Administradores	5
2.2 Autenticação de Funcionários	6
2.3 Autorização (Perfis Administrativos)	6
2.4 Escopo Administrativo	7
3. Padrão de respostas e erros	7
3.1 Formato de sucesso	7
3.2 Formato de erro	7
3.3 Códigos HTTP e erros esperados	8
3.4 Erros específicos de autenticação/autorização	8

4. Endpoints administrativos	9
4.1 Admin Auth (Sessão)	9
4.2 Cargos	9
4.3 Funcionários	10
4.4 Acessos Administrativos	14
4.5 Escolas/Unidades	18
4.6 QR Codes Escolares	22
4.7 Pontos e Relatórios	22
5. Endpoints de autenticação Gov.br (Admin)	25
5.1 Iniciar login	25
5.2 Callback do Gov.br	25
5.3 Logout	26
6. Endpoints de funcionalidade pública (Funcionário)	27
6.1 Login de funcionário	27
6.2 Troca de senha do primeiro acesso	28
6.3 Solicitar recuperação de senha	29
6.4 Validar código de recuperação	29
6.5 Redefinir senha	30
6.6 Registrar ponto (batida)	30
6.7 Obter ponto do dia (dashboard)	32
6.8 Obter histórico de ponto	32
7. Health Check	33
7.1 Status do sistema	33
8. Roteamento de páginas (Admin)	34
9. Configurações de ambiente relevantes	34
Observações sobre o documento original	35

Índice de endpoints

Lista de todos os endpoints documentados, na ordem em que aparecem neste documento. Clique na rota para ir à descrição completa.

Método	Rota	Descrição	Pág.
GET	/api/admin/auth/me	Consulta os dados da sessão administrativa atual e as capacidades do admin.	9
GET	/api/admin/cargos	Lista todos os cargos disponíveis (FUNCIONARIO, INSPETOR, PROFESSOR).	9
GET	/api/admin/funcionarios	Lista funcionário com paginação, filtros e busca por texto.	10
GET	/api/admin/funcionarios/unidades	Lista unidades escolares disponíveis para cadastro de novo funcionário (respeita escopo do admin).	10
GET	/api/admin/funcionarios/:id	Consulta detalhes de um funcionário específico.	11
POST	/api/admin/funcionarios	Cria um novo funcionário.	12
PATCH	/api/admin/funcionarios/:id	Atualiza dados de um funcionário.	13
PATCH	/api/admin/funcionarios/:id/desativar	Desativa um funcionário.	13

PATCH	/api/admin/funcionarios/:id/reactivar	Reativa um funcionário desativado.	14
GET	/api/admin/acessos	Lista acessos administrativos com paginação (respeita escopo).	14
GET	/api/admin/acessos/meu	Lista os próprios acessos do admin autenticado.	15
GET	/api/admin/acessos/capacidades	Lista capacidades por perfil.	16
POST	/api/admin/acessos	Concede um novo acesso administrativo.	16
PATCH	/api/admin/acessos/:id/suspender	Suspende um acesso administrativo.	17
PATCH	/api/admin/acessos/:id/reactivar	Reativa um acesso suspenso.	17
PATCH	/api/admin/acessos/:id/revogar	Revoga um acesso (permanente, não pode ser reativado).	18
GET	/api/admin/escolas	Lista escolas com paginação e filtros (respeita escopo).	18
GET	/api/admin/escolas/diretorias	Lista diretoria de ensino disponíveis para cadastro de nova escola.	19
GET	/api/admin/escolas/cep/:cep	Consulta informações do endereço pelo CEP (BrasilAPI, sem gravar).	19
POST	/api/admin/escolas/preview	Pré-visualiza dados de escola antes de criar (valida DRE dentro do escopo).	20
POST	/api/admin/escolas	Cria uma nova escola/unidade.	20
GET	/api/admin/escolas/:id	Consulta detalhes de uma escola específica.	21
GET	/api/admin/qr-tokens	Obtém o QR Code atual da unidade escolar (link para /ponto/acessar).	22
POST	/api/admin/qr-tokens	Regenera o QR Code da unidade escolar.	22
GET	/api/admin/pontos/hoje	Lista os pontos registrados no dia atual por unidade escolar.	22
GET	/api/admin/pontos/resumo	Obtém resumo consolidado de pontos para o dashboard.	23
GET	/api/admin/pontos/relatorio	Gera relatório diário de pontos.	24
GET	/api/admin/pontos/relatorio/hierarquia	Gera relatório hierárquico por diretoria/unidade.	24
GET	/auth/govbr/login	Redireciona para o Gov.br para autenticação.	25

Método	Rota	Descrição	Pág.
GET	/auth/govbr/callback	Recebe o authorization code do Gov.br e completa o fluxo OAuth 2.0.	25
POST	/auth/govbr/logout	Encerra a sessão admin e redireciona para logout do Gov.br.	26
POST	/ponto/login	Realiza login de funcionário com CPF/email e senha.	27
POST	/ponto/primeiro-acesso/trocar-senha	Permite que funcionário no primeiro acesso troque a senha temporária.	28

POST	/ponto/recuperar-senha/solicitar	Solicita código de recuperação de senha por e-mail.	29
POST	/ponto/recuperar-senha/validar	Valida o código de recuperação enviado por e-mail.	29
POST	/ponto/recuperar-senha/redefinir	Redefine a senha após validar o código de recuperação.	30
POST	/ponto/registrar	Registra a batida de ponto com geolocalização.	30
POST	/ponto/bater	Registra a batida de ponto com geolocalização.	30
GET	/ponto/hoje	Obtém o estado atual do ponto do funcionário no dia (jornada, batidas realizadas, próxima batida).	32
GET	/ponto/historico	Obtém o histórico de batidas do mês atual (ou mês específico).	32
GET	/health	Verifica status de conectividade com banco de dados e Redis (se habilitado).	33

Introdução

Esta documentação descreve os contratos de API do sistema ATestaPonto, um sistema de controle de presença de funcionários em ambiente escolar.

1. Visão geral

A API do ATestaPonto é dividida em dois fluxos principais:

- **API Administrativa** (`/api/admin`): Exclusiva para administradores, com autenticação via Gov.br (Login Único) e controle de perfis/escopos. Todas as rotas exigem sessão administrativa ativa.
- **API de Funcionário** (`/ponto`): Exclusiva para funcionários, com autenticação via JWT próprio (emitido após login com CPF/email e senha). Permite registro de ponto geolocalizado e consulta de histórico.
- **API de Autenticação Gov.br** (`/auth/govbr`): Fluxo OAuth 2.0 com PKCE para autenticação de administradores.

Arquitetura

- **Backend**: Node.js + Express.js
- **Banco de dados**: MySQL
- **Autenticação admin**: Gov.br OAuth 2.0 + sessão server-side (Redis opcional)
- **Autenticação funcionário**: JWT (JSON Web Token) com validação no middleware
- **Escopo administrativo**: Controle granular baseado em perfis (ADMIN_SEDUC, ADMIN_DIRETORIA, DIRETOR, VICE_DIRETOR, SECRETARIA, COORDENADOR) com matriz de delegação

2. Autenticação e autorização

2.1 Autenticação de Administradores

Mecanismo: Sessão Gov.br via OAuth 2.0 com PKCE (Proof Key for Code Exchange)

Fluxo

1. Admin acessa `/auth/govbr/start` ou `/auth/govbr/login` → redireciona para Gov.br
2. Gov.br autentica usuário e redireciona para `/auth/govbr/callback` com `code` e `state`
3. Backend troca `code` por `access_token` no Gov.br (backend-to-backend)
4. Backend busca userinfo no Gov.br e localiza administrador no banco pelo CPF
5. Se encontrado e ativo, cria sessão server-side (`req.session.admin`) com dados do admin
6. Redireciona para `/admin/dashboard`

Sessão admin

```
// Estrutura de sessão (armazenada no Redis ou memória)
{
  authProvider:
  "govbr", id:
  <id_no_banco>, sub:
  "<sub_do_govbr>",
  name: "<nome>",
  email: "<email>",
  loginAt: "<ISO8601>"
}
```

Proteção de rotas: Middleware `ensureAdminApiAuthenticated`

- Valida `req.session.admin` com `authProvider === "govbr"`
- Revalida admin no banco (`admin.ativo === true`)
- Busca e carrega `acessos` do admin (perfis + escopos)
- Popula `req.user`, `req.auth`, `req.acessos`, `req.escopo`, `req.escopoUnidades`

2.2 Autenticação de Funcionários

Mecanismo: JWT (JSON Web Token)

Token

- **Secret:** `JWT_SECRET` (configurado no `.env`)
- **Algoritmo:** RS256 ou HS256 (depende da configuração do `jsonwebtoken`)
- **Payload:**

```
{
  sub: "<funcionario_id>", role: "funcionario",
  credentialVersion: "<hash_baseado_em_senha_data_criacao>"
}
```

- **Expiração:** `FUNCIONARIO_JWT_EXPIRES_IN` (padrão: 20m)

Validação: Middleware `authenticateFuncionario`

- Extrai token de `> Authorization: Bearer <token>`
- Verifica assinatura com `JWT_SECRET`
- Valida `role === "funcionario"`
- Busca funcionário no banco por `sub` + `credentialVersion`
- Verifica `funcionario.ativo === true`
- Verifica `primeiro_acesso === false` (se true, exige troca de senha)

Headers necessários

```
Authorization: Bearer <jwt_token>
```

2.3 Autorização (Perfis Administrativos)

O sistema implementa uma **matriz de delegação** que define quem pode conceder quais perfis:

Concedente	Pode conceder
ADMIN_SEDUC	ADMIN_DIRETORIA, SECRETARIA, COORDENADOR
ADMIN_DIRETORIA	DIRETOR, VICE_DIRETOR, SECRETARIA, COORDENADOR

DIRETOR	SECRETARIA, COORDENADOR
VICE_DIRETOR	SECRETARIA, COORDENADOR
SECRETARIA	(nenhum)
COORDENADOR	(nenhum)

Regras

- Ninguém pode conceder perfil igual ou superior ao próprio
- ADMIN_SEDUC pode suspender/revogar qualquer acesso abaixo de SEDUC
- Demais perfis seguem a matriz de delegação

Capacidades por perfil

Cada perfil possui um conjunto de capacidades que definem o que o administrador pode fazer:

- funcionario.listar, funcionario.criar, funcionario.editar, funcionario.desativar, funcionario.reativar, funcionario.visualizar
- acesso.listar, acesso.conceder, acesso proprio.visualizar
- escola.listar, escola.criar, escola.visualizar
- qr.visualizar, qr.gerar
- ponto.hoje.visualizar, ponto.resumo.visualizar, relatorio.visualizar

2.4 Escopo Administrativo

A autorização é combinada com **escopo territorial**:

- ADMIN_SEDUC: Escopo global (pode acessar tudo)
- ADMIN_DIRETORIA: Escopo limitado às escolas da diretoria de ensino
- DIRETOR, VICE_DIRETOR, SECRETARIA, COORDENADOR: Escopo limitado à unidade escolar específica

O middleware `escopoPorCapacidade(capacidade)`:

- Filtra acessos pelo perfil que possui a capacidade exigida
- Constrói o escopo combinando diretoria/unidade de todos os acessos autorizadores
- Se for ADMIN_DIRETORIA ou escolar, expande unidades permitidas (diretoria → unidades)
- Define `req.escopoUnidades` para uso nos controllers (array de IDs ou `null` para SEDUC)

3. Padrão de respostas e erros

3.1 Formato de sucesso

```
{
  "success": true,
  "data": { /* dados da resposta */ }
}
```

3.2 Formato de erro

```
{
  "success": false,
  "error": {
    "code": "<CODIGO_DE_ERRO>",
    "message": "Mensagem descritiva",
    "details": { /* dados adicionais, se houver */ }
  }
}
```

Obs: Em produção, erros 5xx nunca expõem `message` ou `details` originais; apenas uma mensagem genérica. Logs mantêm os detalhes.

3.3 Códigos HTTP e erros esperados

Código	Erro	Descrição
200	-	Sucesso (geralmente para GETs e PATCHs que não criam recurso)
201	-	Recurso criado com sucesso
400	BAD_REQUEST	Requisição mal formatada (JSON inválido, campo obrigatório ausente, valor inválido)
401	UNAUTHORIZED	Sessão/admin/funcionário não autenticado ou token inválido/expirado
403	FORBIDDEN	Admin/funcionário autenticado mas sem capacidade ou escopo para a operação
404	NOT_FOUND	Recurso solicitado não existe
405	METHOD_NOT_ALLOWED	Método HTTP não permitido para a rota
409	CONFLICT	Conflito de negócio (ex: já 4 batidas hoje, registro duplicado)
422	VALIDATION_ERROR	Erro de validação de campo (ex: CPF inválido, data inválida)
429	RATE_LIMITED	Limite de requisições excedido
500	INTERNAL_ERROR	Erro interno do servidor (logs, mas não exposto ao cliente em produção)
503	DATABASE_UNAVAILABLE / REDIS_UNAVAILABLE	Sistema indisponível (health check)

3.4 Erros específicos de autenticação/autorização

Código	Mensagem	Causa
UNAUTHORIZED	"Sessão administrativa Gov.br obrigatória"	<code>req.session.admin</code> ausente ou <code>authProvider !== "govbr"</code>
UNAUTHORIZED	"Sessão do funcionário inválida"	JWT inválido, assinatura errada ou expirada

UNAUTHORIZED	"Funcionário inexistente ou inativo"	Token válido, mas funcionário não existe ou está desativado
UNAUTHORIZED	"Troca de senha obrigatória antes de acessar o ponto"	Funcionário no primeiro acesso
FORBIDDEN	"Capacidade administrativa insuficiente"	Nenhum acesso do admin possui a capacidade exigida
FORBIDDEN	"Funcionário fora do escopo do administrador"	Funcionário não pertence ao escopo territorial do admin
FORBIDDEN	"Diretoria de ensino fora do escopo do administrador"	DRE não está no escopo do admin
FORBIDDEN	"Unidade escolar fora do escopo do administrador"	Escola não está no escopo do admin

4. Endpoints administrativos

Todas as rotas `/api/admin/*` exigem autenticação Gov.br e são protegidas por `ensureAdminApiAuthenticated`.

4.1 Admin Auth (Sessão)

GET	/api/admin/auth/me
Descrição	Consulta os dados da sessão administrativa atual e as capacidades do admin.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Nenhuma específica (qualquer admin autenticado pode consultar).

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "admin": {
      "id": 1,
      "sub": "<govbr_sub>",
      "name": "Nome do Admin",
      "email": "admin@exemplo.com",
      "loginAt": "2024-10-06T10:30:00.000Z"
    },
    "capacidades": [
      "funcionario.listar",
      "funcionario.criar",
      "escola.listar",
      "acesso.listar"
    ]
  }
}
```

Erros esperados

Descrição	Lista todos os cargos disponíveis (FUNCIONARIO, INSPETOR, PROFESSOR).
Autenticação	Sessão Gov.br obrigatória.
Autorização	Nenhuma específica (qualquer admin pode listar).
Parâmetros	Nenhum.

HTTP	Código	Mensagem / condição
401	UNAUTHORIZED	Sessão administrativa não autenticada

4.2 Cargos

GET `/api/admin/cargos`

Response de sucesso (200)

```
{
  "success": true,
  "data": ["FUNCIONARIO", "INSPETOR", "PROFESSOR"]
}
```

Erros esperados

- Nenhum específico (se a consulta falhar, cai no middleware de erro genérico)

4.3 Funcionários

GET `/api/admin/funcionarios`

Descrição Lista funcionário com paginação, filtros e busca por texto.

Autenticação Sessão Gov.br obrigatória.

Autorização Requer capacidade `funcionario.listar` em algum acesso ativo.

Query Parameters

Campo	Obrigatório	Tipo	Descrição
<code>page</code>	Não	integer	Número da página (≥ 1), padrão: 1
<code>limit</code>	Não	integer	Itens por página (1-100), padrão: 20
<code>ativo</code>	Não	string	Filtro por estado: <code>"true"</code> ou <code>"false"</code>

cargo	Não	string	Filtro por cargo: FUNCIONARIO, INSPETOR, PROFESSOR
q	Não	string	Busca por nome, CPF ou email (max 120 chars)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "nome": "Nome do Funcionário",
        "cpf": "000.000.000-00",
        "email": "func@exemplo.com",
        "cargo": "FUNCIONARIO",
        "ativo": true,
        "unidade_escolar_id": 1,
        "unidade_escolar_nome": "Escola Exemplo",
        "diretoria_ensino_id": 1,
        "diretoria_ensino_nome": "Diretoria de Ensino Exemplo"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
    }
  }
}
```

GET /api/admin/funcionarios/unidades

Descrição Lista unidades escolares disponíveis para cadastro de novo funcionário (respeita escopo do admin).

Autenticação Sessão Gov.br obrigatória.

Autorização Requer capacidade funcionario.criar.

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "nome": "Escola Exemplo",
        "codigo": "000001",
        "diretoria_ensino_id": 1,
        "diretoria_ensino_nome": "Diretoria de Ensino Exemplo"
      }
    ]
  }
}
```

GET /api/admin/funcionarios/:id

Descrição Consulta detalhes de um funcionário específico.

Autenticação Sessão Gov.br obrigatória.

Autorização Path	• Requer capacidade <code>funcionario.visualizar</code>
Parameters	• Funcionário deve estar dentro do escopo do admin

Campo	Descrição
<code>id</code>	ID do funcionário (inteiro >= 1)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Nome do Funcionário",
    "cpf": "000.000.000-00",
    "email": "func@exemplo.com",
    "telefone": "11999999999",
    "cargo": "FUNCIONARIO",
    "entrada": "08:00:00",
    "saida_almoco": "12:00:00",
    "retorno_almoco": "13:00:00",
    "saida": "17:00:00",
    "ativo": true,
    "unidade_escolar_id": 1,
    "unidade_escolar_nome": "Escola Exemplo",
    "diretoria_ensino_id": 1,
    "diretoria_ensino_nome": "Diretoria de Ensino Exemplo"
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
403	FORBIDDEN	"Funcionário sem vínculo visível ao escopo do administrador"
404	NOT_FOUND	Funcionário não encontrado (não revela se existe fora do escopo)

POST `/api/admin/funcionarios`

Descrição Cria um novo funcionário.

Autenticação Sessão Gov.br obrigatória.

Autorização Requer capacidade `funcionario.criar`.

Request Body

```
{
  "nome": "Nome do Funcionário",
  "cpf": "00000000000",
  "email": "func@exemplo.com",
  "cargo": "FUNCIONARIO",
  "unidade_escolar_id": 1,
  "entrada": "08:00:00",
  "saida_almoco": "12:00:00",
  "retorno_almoco": "13:00:00",
  "saida": "17:00:00",
  "telefone": "11999999999",
  "ativo": true
}
```

Validações

- **nome:** 3-55 caracteres, sem caracteres < e >
- **cpf:** 11 dígitos, formato numérico
- **email:** formato válido de email
- **cargo:** FUNCIONARIO, INSPETOR ou PROFESSOR
- **unidade_escolar_id:** inteiro >= 1 (deve existir e estar no escopo do admin)
- **ativo:** booleano (true, false, 1, 0, "true", "false")
- **Horários:** formato HH:mm:ss, ordem obrigatória: entrada < saida_almoco < retorno_almoco < saida
- **telefone:** 10 ou 11 dígitos (opcional)

Response de sucesso (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Nome do Funcionário",
    "cpf": "000.000.000-00",
    "email": "func@exemplo.com",
    "cargo": "FUNCIONARIO",
    "ativo": true,
    "unidade_escolar_id": 1,      "unidade_escolar_nome": "Escola Exemplo"
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
403	FORBIDDEN	"Diretoria de ensino fora do escopo do administrador"
403	FORBIDDEN	"Unidade escolar fora do escopo do administrador"
409	CONFLICT	"Registro duplicado de ponto detectado" (se CPF já existir)

PATCH**/api/admin/funcionarios/:id**

Descrição Atualiza dados de um funcionário.

Autenticação Sessão Gov.br obrigatória.

Autorização Path • Requer capacidade `funcionario.editar`
Parameters • Funcionário deve estar dentro do escopo do admin

Campo	Descrição
id	ID do funcionário

Request Body (campos opcionais)

```
{
  "nome": "Nome Atualizado",
  "email": "func.atualizado@exemplo.com",
  "telefone": "11988888888",
  "cargo": "INSPETOR",
  "entrada": "08:30:00",
  "saida_almoco": "12:00:00",
  "retorno_almoco": "13:00:00",
  "saida": "17:30:00"
}
```

```
}

```

Validações

- Todos os campos são opcionais
- Validações idênticas ao POST (exceto `unidade_escolar_id` não pode ser atualizada aqui)
- Horários: ordem obrigatória

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Nome Atualizado",
    "email": "func.atualizado@exemplo.com",
    "cargo": "INSPETOR",
    "ativo": true,
    "unidade_escolar_id": 1
  }
}
```

PATCH
/api/admin/funcionarios/:id/desativar

Descrição Desativa um funcionário.

Autenticação Sessão Gov.br obrigatória.

Autorização

- Requer capacidade `funcionario.desativar`
- Funcionário deve estar dentro do escopo do admin

Request Body

```
{
  "confirmacao": "DESATIVAR"
}
```

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Nome do Funcionário",
    "ativo": false
  }
}
```

PATCH
/api/admin/funcionarios/:id/reactivar

Descrição Reativa um funcionário desativado.

Autenticação Sessão Gov.br obrigatória.

Autorização

- Requer capacidade `funcionario.reativar`
- Funcionário deve estar dentro do escopo do admin

Request Body

```
{
  "confirmacao": "REATIVAR"
}
```

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Nome do Funcionário",
    "ativo": true
  }
}
```

4.4 Acessos Administrativos

GET	/api/admin/acessos
-----	--------------------

Descrição	Lista acessos administrativos com paginação (respeita escopo).
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>acesso.listar</code> .

Query Parameters

Campo	Obrigatório	Tipo	Descrição
page	Não	integer	Número da página (>= 1)
limit	Não	integer	Itens por página (1-100)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "cpf": "000.000.000-00",
        "nome": "Nome do Admin",
        "email": "admin@exemplo.com",
        "perfil": "DIRETOR",
        "unidade_escolar_id": 1,
        "unidade_escolar_nome": "Escola Exemplo",
        "status": "ATIVO",
        "data_inicio": "2024-01-01",
        "data_fim": null,
        "criado_em": "2024-01-01T10:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}
```

GET	/api/admin/acessos/meu
-----	------------------------

Descrição	Lista os próprios acessos do admin autenticado.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>acesso proprio visualizar</code> .

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "perfil": "DIRETOR",
        "unidade_escolar_id": 1,
        "unidade_escolar_nome": "Escola Exemplo",
        "status": "ATIVO",
        "data_inicio": "2024-01-01",
        "data_fim": null
      }
    ]
  }
}
```

GET

/api/admin/acessos/capacidades

Descrição	Lista capacidades por perfil.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>acesso proprio visualizar</code> .

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "perfis": [
      {
        "perfil": "ADMIN_SEDUC",
        "capacidades": ["funcionario.listar", "funcionario.criar", "funcionario.editar",
          "funcionario.desativar", "funcionario.reativar", "funcionario.visualizar",
          "acesso.listar", "acesso.conceder", "acesso proprio visualizar", "escola.listar",
          "escola.criar", "escola.visualizar", "qr.visualizar", "qr.gerar",
          "ponto.hoje.visualizar", "ponto.resumo.visualizar", "relatorio.visualizar"]
      },
      {
        "perfil": "DIRETOR",
        "capacidades": ["funcionario.listar", "funcionario.visualizar", "escola.visualizar",
          "qr.visualizar", "ponto.hoje.visualizar", "ponto.resumo.visualizar",
          "relatorio.visualizar"]
      }
    ]
  }
}
```

POST

/api/admin/acessos

Descrição	Concede um novo acesso administrativo.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>acesso conceder</code> .

Request Body

```
{
  "cpf": "00000000000",
  "nome": "Nome do Admin",
  "email": "admin@exemplo.com",
  "perfil": "DIRETOR",
  "unidade_escolar_id": 1,
  "status": "ATIVO",
  "data_inicio": "2024-01-01",
  "data_fim": null
}
```

Validações

- `cpf`: 11 dígitos (se admin ainda não existe no sistema)
- `nome`: 3-150 caracteres (obrigatório se admin não existe)
- `email`: formato válido (opcional)
- `perfil`: ADMIN_SEDUC, ADMIN_DIRETORIA, DIRETOR, VICE_DIRETOR, SECRETARIA, COORDENADOR
- `unidade_escolar_id`: inteiro >= 1 (obrigatório para perfis escolares e ADMIN_DIRETORIA)
- `diretoria_ensino_id`: inteiro >= 1 (obrigatório para ADMIN_DIRETORIA, proibido para outros)
- `status`: ATIVO ou SUSPENSO
- `data_inicio`, `data_fim`: formato ISO8601

Restrições

- Regra de matriz de delegação (quem pode conceder qual perfil)
- Regra de escopo (quem pode conceder a qual DRE/unidade)

Response de sucesso (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "cpf": "000.000.000-00",
    "nome": "Nome do Admin",
    "perfil": "DIRETOR",
    "unidade_escolar_id": 1,
    "status": "ATIVO",
    "data_inicio": "2024-01-01"
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
403	FORBIDDEN	"Capacidade administrativa insuficiente"
409	CONFLICT	"Registro duplicado" (se CPF já tiver acesso ativo)
PATCH	/api/admin/acessos/:id/suspender	
Descrição	Suspende um acesso administrativo.	
Autenticação	Sessão Gov.br obrigatória.	

- Autorização Request**
- Requer capacidade `acesso.conceder`
- Body**
- O admin deve poder suspender o perfil-alvo (matriz de delegação)

```
{ }
```

(Sem corpo necessário)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "status": "SUSPENSO"
  }
}
```

PATCH
/api/admin/acessos/:id/reactivar

Descrição	Reativa um acesso suspenso.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Mesma regra de suspender.

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "status": "ATIVO"
  }
}
```

PATCH
/api/admin/acessos/:id/revogar

Descrição Revoga um acesso (permanente, não pode ser reativado).

Autenticação Sessão Gov.br obrigatória.

Autorização Mesma regra de suspender.

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "status": "REVOGADO"
  }
}
```

4.5 Escolas/Unidades

GET
/api/admin/escolas

Descrição Lista escolas com paginação e filtros (respeita escopo).

Autenticação Sessão Gov.br obrigatória.

Autorização Requer capacidade `escola.listar`.

Query Parameters

Campo	Obrigatório	Tipo	Descrição
<code>page</code>	Não	integer	Número da página (≥ 1)
<code>limit</code>	Não	integer	Itens por página (1-100)
<code>ativa</code>	Não	string	Filtro por estado: <code>"true"</code> ou <code>"false"</code>
<code>diretoria_ensino_id</code>	Não	integer	Filtro por DRE
<code>q</code>	Não	string	Busca por nome ou código (max 120 chars)

```
{
  "success": true,
  "data": {
    "items": [
      {
```

Response de sucesso (200)

```
{
  "id": 1,
  "nome": "Escola Exemplo",
  "codigo": "000001",
  "ativo": true,
  "diretoria_ensino_id": 1,
  "diretoria_ensino_nome": "Diretoria de Ensino
Exemplo",
  "logradouro": "Rua das Escolas",
  "numero": "123",
  "bairro": "Bairro Exemplo",
  "cep": "00000000"
},
{
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

GET	/api/admin/escolas/diretorias
Descrição	Lista diretoria de ensino disponíveis para cadastro de nova escola.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>escola.criar</code> .

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "nome": "Diretoria de
Ensino Exemplo",
        "codigo":
"DE001"
      }
    ]
  }
}
```

GET	/api/admin/escolas/cep/:cep
Descrição	Consulta informações do endereço pelo CEP (BrasilAPI, sem gravar).
Autenticação	Sessão Gov.br obrigatória.
Autorização Path	Requer capacidade <code>escola.criar</code> .

Parameters

Campo	Descrição
-------	-----------

Resposta de sucesso (200)

cep	CEP com 8 dígitos numéricos
<pre>{ "success": true, "data": { "cep": "00000000", "logradouro": "Rua das Escolas", "complemento": "", "bairro": "Bairro Exemplo", "localidade": "Cidade Exemplo", "uf": "SP", "ddd": "11" } }</pre>	

POST	/api/admin/escolas/preview
Descrição	Pré-visualiza dados de escola antes de criar (valida DRE dentro do escopo).
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>escola.criar</code> .

Request Body

```
{
  "nome": "Escola Exemplo",
  "codigo": "000001",
  "diretoria_ensino_id": 1,
  "logradouro": "Rua das Escolas",
  "numero": "123",
  "bairro": "Bairro Exemplo",
  "cep": "00000000"
}
```

Resposta de sucesso (200)

```
{
  "success": true,
  "data": {
    "nome": "Escola Exemplo",
    "codigo": "000001",
    "diretoria_ensino_id": 1,
    "diretoria_ensino_nome": "Diretoria de Ensino Exemplo",
    "logradouro": "Rua das Escolas",
    "numero": "123",
    "bairro": "Bairro Exemplo",
    "cep": "00000000"
  }
}
```

POST	/api/admin/escolas
Descrição	Cria uma nova escola/unidade.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade <code>escola.criar</code> .

Request Body

```
{
  "nome": "Escola Exemplo",
  "codigo": "000001",
  "diretoria_ensino_id": 1,
  "logradouro": "Rua das Escolas",
  "numero": "123",
  "bairro": "Bairro Exemplo",
  "cep": "00000000"
}
```

Response de sucesso (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Escola Exemplo",
    "codigo": "000001",
    "ativo": true,
    "diretoria_ensino_id": 1
  }
}
```

GET

/api/admin/escolas/:id

Descrição

Consulta detalhes de uma escola específica.

Autenticação

Sessão Gov.br obrigatória.

Autorização Path

- Requer capacidade `escola.visualizar`

Parameters

- Escola deve estar dentro do escopo do admin

Campo	Descrição
id	ID da escola

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "nome": "Escola Exemplo",
    "codigo": "000001",
    "ativo": true,
    "diretoria_ensino_id": 1,
    "diretoria_ensino_nome": "Diretoria de Ensino Exemplo",
    "logradouro": "Rua das Escolas",
    "numero": "123",
    "bairro": "Bairro Exemplo",
    "cep": "00000000",
    "latitude": -23.55,
    "longitude": -46.63,
    "raio_permitido_metros": 100
  }
}
```

4.6 QR Codes Escolares

GET

/api/admin/qr-tokens

Descrição Obtém o QR Code atual da unidade escolar (link para [/ponto/acessar](#)).

Autenticação Sessão Gov.br obrigatória.

Autorização

- Requer capacidade `qr.visualizar`
- Unidade deve estar associada a um acesso do admin

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "qrCode": "https://dominio/ponto/acessar"
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
409	CONFLICT	"Operação de QR ambígua: mais de uma unidade escolar elegível" (quando admin tem múltiplos acessos com <code>qr.visualizar</code> e cada um aponta para unidade diferente)

POST

/api/admin/qr-tokens

Descrição Regenera o QR Code da unidade escolar.

Autenticação Sessão Gov.br obrigatória.

Autorização Request Body

- Requer capacidade `qr.gerar`
- Unidade deve estar associada a um acesso do admin

```
{ }
```

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "qrCode": "https://dominio/ponto/acessar?regenerado=true"
  }
}
```

4.7 Pontos e Relatórios

GET

/api/admin/pontos/hoje

Descrição Lista os pontos registrados no dia atual por unidade escolar.

Autenticação Sessão Gov.br obrigatória.

Autorização Requer capacidade `ponto.hoje.visualizar`.

Query Parameters

Campo	Obrigatório	Tipo	Descrição
data	Não	string	Data no formato YYYY-MM-DD (padrão: hoje)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "data_referencia": "2024-10-06",
    "unidades": [
      {
        "id": 1,
        "nome": "Escola Exemplo",
        "total_funcionarios": 50,
        "total_registros": 180,
        "registros": [
          {
            "funcionario_id": 1,
            "funcionario_nome": "Nome do Funcionário",
            "funcao": "DIRETOR",
            "entrada": "08:00:00",
            "saida_almoco": "12:00:00",
            "retorno_almoco": "13:00:00",
            "saida": "17:00:00",
            "status": "COMPLETO"
          }
        ]
      }
    ]
  }
}
```

GET	/api/admin/pontos/resumo
Descrição	Obtém resumo consolidado de pontos para o dashboard.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Requer capacidade ponto.resumo.visualizar.

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "total_funcionarios": 500,
    "total_unidades": 20,
    "total_diretorias": 3,
    "registros_hoje": {
      "total_registros": 1800,
      "jornadas_completas": 1750,
      "jornadas_incompletas": 50
    }
  }
}
```

GET	/api/admin/pontos/relatorio
Descrição	Gera relatório diário de pontos.
Autenticação	Sessão Gov.br obrigatória.

Autorização

Requer capacidade relatorio.visualizar.

Query Parameters

Campo	Obrigatório	Tipo	Descrição
data	Não	string	Data no formato YYYY-MM-DD (padrão: hoje)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "data_referencia": "2024-10-06",
    "unidades": [
      {
        "id": 1,
        "nome": "Escola Exemplo",
        "total_funcionarios": 50,
        "registros": [
          {
            "funcionario_id": 1,
            "funcionario_nome": "Nome do Funcionário",
            "funcao": "DIRETOR",
            "entrada": "08:00:00",
            "saida_almoco": "12:00:00",
            "retorno_almoco": "13:00:00",
            "saida": "17:00:00",
            "status": "COMPLETO"
          }
        ]
      }
    ],
    "resumo": {
      "total_funcionarios": 500,
      "total_registros": 1800,
      "jornadas_completas": 1750,
      "jornadas_incompletas": 50
    }
  }
}
```

GET

/api/admin/pontos/relatorio/hierarquia

Descrição

Gera relatório hierárquico por diretoria/unidade.

Autenticação

Sessão Gov.br obrigatória.

Autorização

Requer capacidade relatorio.visualizar.

Query Parameters

Campo	Obrigatório	Tipo	Descrição
data	Não	string	Data no formato YYYY-MM-DD (padrão: hoje)
diretoria_ensino_id	Não	integer	Filtro por DRE
unidade_escolar_id	Não	integer	Filtro por escola

Response de sucesso (200)

```
{
  "success": true,
  "data": {
```

```
"data_referencia": "2024-10-06",
"diretorias": [
  {
    "id": 1,
    "nome": "Diretoria de Ensino Exemplo",
    "unidades": [
      {
        "id": 1,
        "nome": "Escola Exemplo",
        "registros": [...]
      }
    ],
    "resumo": {...}
  },
  "resumo_geral": {...}
}
```

5. Endpoints de autenticação Gov.br (Admin)

5.1 Iniciar login

GET	/auth/govbr/login
Descrição	Redireciona para o Gov.br para autenticação.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Fluxo

- 1. Backend gera `state` (anti-CSRF) e `codeVerifier/codeChallenge` (PKCE)
- 2. Armazena na sessão (`req.session.oauthGovbr`)
- 3. Redireciona para URL de autorização do Gov.br

Response: HTTP 302 (redirect para Gov.br)

5.2 Callback do Gov.br

GET	/auth/govbr/callback
Descrição	Recebe o authorization code do Gov.br e completa o fluxo OAuth 2.0.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Query Parameters

Campo	Descrição
-------	-----------

<code>code</code>	Authorization code retornado pelo Gov.br
<code>state</code>	State anti-CSRF (deve bater com o gerado no <code>/login</code>)
<code>error</code>	(opcional) Mensagem de erro se autenticação falhou

Fluxo

1. Valida `state` contra sessão
2. Troca `code` por `access_token` no Gov.br (backend-to-backend)
3. Busca userinfo do Gov.br com o access token
4. Localiza admin no banco pelo CPF do userinfo
5. Se encontrado e ativo, cria sessão admin
6. Registra último login
7. Redireciona para `/admin/dashboard`

Response: HTTP 302 (redirect para `/admin/dashboard` ou página de erro)

Erros esperados

HTTP	Código	Mensagem / condição
400	<code>BAD_REQUEST</code>	"Access token não é aceito no callback" (tentativa de implicit flow)
400	<code>BAD_REQUEST</code>	"Callback Gov.br sem dados de autenticação válidos"
401	<code>UNAUTHORIZED</code>	"State Gov.br inválido"
403	<code>FORBIDDEN</code>	"Usuário Gov.br não cadastrado como administrativo"
403	<code>FORBIDDEN</code>	"Administrativo inativo"

5.3 Logout

POST	<code>/auth/govbr/logout</code>
Descrição	Encerra a sessão admin e redireciona para logout do Gov.br.
Autenticação	Sessão Gov.br obrigatória.
Autorização	Nenhuma específica.

Headers

- `Origin` ou `Referer`: Deve ser same-origin (mesmo domínio/porta)

Response

```
{ }
```

(Sem corpo, redireciona via `Location` header ou JavaScript no frontend)

6. Endpoints de funcionalidade pública (Funcionário)

6.1 Login de funcionário

POST

/ponto/login

Descrição	Realiza login de funcionário com CPF/email e senha.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Request Body

```
{
  "identificador": "00000000000",
  "senha": "minha_senha_8 caracteres"
}
```

Ou:

```
{
  "identificador": "funcionario@exemplo.com",
  "senha": "minha_senha_8 caracteres"
}
```

Validações

- `identificador`: CPF (11 dígitos) ou email válido
- `senha`: texto, 8-72 caracteres

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "token": "<jwt_token>",
    "expiresIn": "20m",
    "primeiro_acesso": false,
    "funcionario": {
      "id": 1,
      "nome": "Nome do Funcionário",
      "email": "func@exemplo.com",
      "cpf": "000.000.000-00"
    }
  }
}
```

Response para primeiro acesso (200)

```
{
  "success": true,
  "data": {
    "primeiro_acesso": true,
    "token_primeiro_acesso": "<jwt_token_temporario>",
    "expiresIn": "10m",
    "funcionario": {
      "id": 1,
      "nome": "Nome do Funcionário",
      "email": "func@exemplo.com",
      "cpf": "000.000.000-00"
    }
  }
}
```

(O funcionário deve usar este token para trocar senha em `/ponto/primeiro-acesso/trocar-senha`)

Erros esperados

HTTP	Código	Mensagem / condição
------	--------	---------------------

400	BAD_REQUEST	"CPF/email ou senha inválidos" (cpf inválido, email inválido, senha curta)
401	UNAUTHORIZED	"CPF/email ou senha inválidos" (credenciais erradas)
401	UNAUTHORIZED	"CPF/email ou senha inválidos" (funcionário sem vínculo ativo)
403	FORBIDDEN	"QR Code não corresponde à unidade escolar do funcionário"

6.2 Troca de senha do primeiro acesso

POST	/ponto/primeiro-acesso/trocar-senha
Descrição	Permite que funcionário no primeiro acesso troque a senha temporária.
Autenticação	Token temporário de primeiro acesso via <code>Authorization: Bearer.</code>
Autorização	Nenhuma específica (qualquer funcionário com token válido pode trocar).

Request Body

```
{
  "novaSenha": "nova_senha_8_caracteres"
}
```

Validações

- `novaSenha`: texto, 8-72 caracteres

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "message": "Senha trocada com sucesso"
  }
}
```

6.3 Solicitar recuperação de senha

POST	/ponto/recuperar-senha/solicitar
Descrição	Solicita código de recuperação de senha por e-mail.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Request Body

```
{
  "cpf": "00000000000"
}
```

Validações

- `cpf`: 11 dígitos, válido

Response de sucesso (200)

```
{
  "success": true,
  "data": {
```

```
{
  "message": "Código de recuperação enviado para o e-mail"
}
```

6.4 Validar código de recuperação

POST	/ponto/recuperar-senha/validar
Descrição	Valida o código de recuperação enviado por e-mail.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Request Body

```
{
  "cpf": "00000000000",
  "codigo": "123456"
}
```

Validações

- **cpf**: 11 dígitos, válido
- **codigo**: 6 dígitos numéricos

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "message": "Código válido"
  }
}
```

6.5 Redefinir senha

POST	/ponto/recuperar-senha/redefinir
Descrição	Redefine a senha após validar o código de recuperação.
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Request Body

```
{
  "cpf": "00000000000",
  "codigo": "123456",
  "novaSenha": "nova_senha_8_caracteres"
}
```

Validações

- **cpf**: 11 dígitos, válido
- **codigo**: 6 dígitos numéricos
- **novaSenha**: texto, 8-72 caracteres

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "message": "Senha redefinida com sucesso"
  }
}
```

6.6 Registrar ponto (batida)

Descrição Registra a batida de ponto com geolocalização.

Autenticação JWT de funcionário via `Authorization: Bearer`.

Autorização Nenhuma específica (qualquer funcionário autenticado pode bater ponto).

POST `/ponto/registrar`

POST ou `/ponto/bater`

Request Body

```
{
  "chaveIdempotencia": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "latitude": -23.55,
  "longitude": -46.63,
  "accuracy": 50,
  "timestamp": 1728213600000
}
```

Validações

- `chaveIdempotencia`: UUID v4 (obrigatório para evitar duplicação)
- `latitude`: número entre -90 e 90
- `longitude`: número entre -180 e 180
- `accuracy`: número positivo (máximo 100 metros)
- `timestamp`: inteiro positivo (milissegundos)

Fluxo de validação

1. Backend resolve DRE/unidade do funcionário via vínculo ativo
2. Busca geolocalização da escola (`latitude`, `longitude`, `raio_permitido_metros`)
3. Calcula distância entre posição do funcionário e escola
4. Verifica se está dentro do raio permitido
5. Verifica hora atual na zona de São Paulo
6. Determina qual batida é (1ª=Entrada, 2ª=Saída Almoço, etc.)
7. Registra na tabela `ponto` com locking para evitar corridas
8. Salva idempotência para replay

Response de sucesso (201)

```
{
  "success": true,
  "data": {
    "ponto": {
      "id": 1,

```

```
    "sequencia": 1,
    "tipo": "ENTRADA",
    "registrado_em": "2024-10-06 08:00:00",
    "distancia_metros": 12.5
  },
  "funcionario": {
    "id": 1,
    "nome": "Nome do Funcionário",
    "email": "func@exemplo.com",
    "cpf": "000.000.000-00"
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
400	<code>BAD_REQUEST</code>	"Localização inválida para registro de ponto" (coordenadas fora de formato)
400	<code>BAD_REQUEST</code>	"Leitura GPS exige precisão maior que zero e de até 100 metros, com timestamp em milissegundos."
400	<code>BAD_REQUEST</code>	"Leitura GPS vencida ou com horário futuro. Atualize a localização e confira o relógio do aparelho."
401	<code>UNAUTHORIZED</code>	"Sessão do funcionário inválida" (JWT inválido/expirado)
403	<code>FORBIDDEN</code>	"Funcionário inativo"
403	<code>FORBIDDEN</code>	"A localização e sua margem de precisão devem estar dentro da área permitida da escola."
409	<code>CONFLICT</code>	"Funcionário já realizou 4 batidas hoje"
409	<code>CONFLICT</code>	"Chave de idempotência já usada com outros dados"

6.7 Obter ponto do dia (dashboard)

GET	/ponto/hoje
Descrição	Obtém o estado atual do ponto do funcionário no dia (jornada, batidas realizadas, próxima batida).
Autenticação	JWT de funcionário via <code>Authorization: Bearer.</code>
Autorização	Nenhuma específica.
Query Parameters:	Nenhum.
Response de sucesso (200)	

```
{
  "success": true,
  "data": {
    "funcionario": {
      "nome": "Nome do Funcionário",
      "cargo": "DIRETOR"
    },
    "jornada": {
      "entrada": "08:00:00",
      "saida_almoco": "12:00:00",
      "retorno_almoco": "13:00:00",
      "saida": "17:00:00"
    }
  }
}
```



```
    },
    "ponto": {
      "data_referencia": "2024-10-06",
      "entrada": "08:00:00",
      "saida_almoco": "12:00:00",
      "retorno_almoco": null,
      "saida": null
    },
    "proxima_batida": "SAIDA_ALMOCO",
    "jornada_concluida": false
  }
}
```

Erros esperados

HTTP	Código	Mensagem / condição
401	UNAUTHORIZED	"Sessão do funcionário inválida"
404	NOT_FOUND	"Funcionário sem vínculo ativo"
404	NOT_FOUND	"Funcionário não encontrado"
403	FORBIDDEN	"Funcionário inativo"

6.8 Obter histórico de ponto

GET	/ponto/historico
Descrição	Obtém o histórico de batidas do mês atual (ou mês específico).
Autenticação	JWT de funcionário via <code>Authorization: Bearer.</code>
Autorização	Nenhuma espe

Query Parameters

Campo	Obrigatório	Tipo	Descrição
mes	Não	string	Mês no formato <code>YYYY-MM</code> (padrão: mês atual)

Response de sucesso (200)

```
{
  "success": true,
  "data": {
    "periodo": "2024-10",
    "resumo": {
      "dias_com_registro": 15,
      "jornadas_completas": 12,
      "jornadas_incompletas": 3,
      "total_minutos": 600
    },
    "registros": [
      {
        "data_referencia": "2024-10-06",
        "entrada": "08:00:00",
        "saida_almoco": "12:00:00",
        "retorno_almoco": "13:00:00",
        "saida": "17:00:00",
        "total_minutos": 480,
        "status": "COMPLETO",
        "vinculo_funcional_id": 1,
        "unidade_escolar_id": 1,
        "unidade_escolar_nome": "Escola Exemplo"
      }
    ]
  }
}
```

```

    }
  ]
}
}

```

Erros esperados

HTTP	Código	Mensagem / condição
400	<code>BAD_REQUEST</code>	"mes deve usar o formato YYYY-MM"
401	<code>UNAUTHORIZED</code>	"Sessão do funcionário inválida"

7. Health Check

7.1 Status do sistema

GET	/health
Descrição	Verifica status de conectividade com banco de dados e Redis (se habilitado).
Autenticação	Nenhuma (público).
Autorização	Nenhuma.

Response de saúde (200)

```

{
  "success": true,
  "data": {
    "status": "ok",
    "database": "connected",
    "redis": "connected"
  }
}

```

Response com falha (503)

```

{
  "success": false,
  "data": {
    "status": "unhealthy",
    "database": "unavailable",
    "redis": "connected"
  },
  "error": {
    "code": "DATABASE_UNAVAILABLE",
    "message": "Banco de dados indisponível."
  }
}

```

8. Roteamento de páginas (Admin)

O sistema também fornece rotas HTML para a interface administrativa, servidas via `sendView()` com headers de cache-control para evitar cache de conteúdo dinâmico:

Rota	Descrição
GET /admin/dashboard	Dashboard principal
GET /admin/funcionarios	Lista de funcionários
GET /admin/acessos	Gerenciamento de acessos
GET /admin/escolas	Lista de escolas
GET /admin/qr	QR Codes

Todas essas rotas exigem sessão admin válida; caso contrário, redirecionam para </auth/govbr/login>.

9. Configurações de ambiente relevantes

As seguintes variáveis de ambiente afetam diretamente o comportamento da API:

Variável	Obrigatória	Descrição
JWT_SECRET	Sim	Segredo para assinar tokens de funcionário (mínimo 32 chars, misto de caracteres)
SESSION_SECRET	Sim	Segredo para sessão admin (diferente do JWT_SECRET)
ADMIN_SESSION_TTL_MS	Não	Tempo de vida da sessão admin em ms (padrão: 2h)
FUNCIONARIO_JWT_EXPIRES_IN	Não	Tempo de validade do JWT de funcionário (padrão: 20m)
FIRST_ACCESS_TOKEN_EXPIRES_IN	Não	Tempo de validade do token de primeiro acesso (padrão: 10m)
GOVBR_AUTHORIZE_URL	Sim	URL de autorização do Gov.br (HTTPS)
GOVBR_TOKEN_URL	Sim	URL de troca de code por token (HTTPS)
GOVBR_USERINFO_URL	Sim	URL de busca de userinfo (HTTPS)
GOVBR_CLIENT_ID	Sim	Client ID do Gov.br
GOVBR_CLIENT_SECRET	Sim	Client Secret do Gov.br
ADMIN_GOVBR_SUBS	Sim (pelo menos um)	Lista de <code>sub</code> do Gov.br permitidos como admin
ADMIN_GOVBR_EMAILS	Sim (pelo menos um, se não houver SUBS)	Lista de e-mails permitidos como admin
Variável	Obrigatória	Descrição
SCHOOL_UNIT_CODE	Não	Código da escola (padrão: "DEFAULT")
SCHOOL_LATITUDE	Não	Latitude da escola (para geofencing)
SCHOOL_LONGITUDE	Não	Longitude da escola (para geofencing)
ALLOWED_RADIUS_METERS	Não	Raio permitido para bater ponto (padrão: 100m)

SPRING_BACKEND_URL	Não	URL do backend Spring (se houver integração)
--------------------	-----	--

Nota: Esta documentação foi gerada exclusivamente com base no código atual do projeto ATestaPonto. Nenhuma informação foi inventada ou extrapolada sem comprovação direta no código.

Observações sobre o documento original

Os pontos abaixo foram identificados durante a revisão do **API.md**. O conteúdo técnico foi preservado exatamente como está no original; esta seção apenas os registra para eventual conferência.

Pontos de atenção na documentação original

Referência	Observação
Seção 1	O texto afirma que a API é dividida em "dois fluxos principais", mas lista três (<code>/api/admin</code> , <code>/ponto</code> e <code>/auth/govbr</code>).
Seção 2.1	Cita <code>/auth/govbr/start</code> e <code>/auth/govbr/login</code> como pontos de entrada do fluxo; somente <code>/auth/govbr/login</code> é documentada como endpoint (seção 5.1).
Seções 4.2, 4.7 e 6.7	<code>GET /api/admin/cargos</code> lista os cargos <code>FUNCIONARIO</code> , <code>INSPETOR</code> , <code>PROFESSOR</code> , mas os exemplos de <code>GET /api/admin/pontos/hoje</code> , <code>GET /api/admin/pontos/relatorio</code> campo <code>funcao</code> e <code>GET /ponto/hoje</code> (campo <code>cargo</code>) exibem o valor <code>DIRETOR</code> .
Seção 4.3	Em <code>POST /api/admin/funcionarios</code> , o erro 409 para CPF já existente traz a mensagem "Registro duplicado de ponto detectado", que menciona "ponto".
Seção 6	O título da seção é "Endpoints de funcionalidade pública", porém <code>POST /ponto/registrar</code> , <code>GET /ponto/hoje</code> e <code>GET /ponto/historico</code> exigem JWT de funcionário.
Seção 6.1	O erro 403 "QR Code não corresponde à unidade escolar do funcionário" está documentado em <code>POST /ponto/login</code> , mas o corpo da requisição não inclui campo relacionado a QR Code.
Seção 2.2 e 9	O algoritmo do JWT é descrito como "RS256 ou HS256"; apenas <code>JWT_SECRET</code> é documentado como configuração de assinatura.
Seção 4.7	O exemplo de <code>GET /api/admin/pontos/relatorio/hierarquia</code> usa reticências (<code>[...]</code> , <code>{...}</code>) como abreviação; mantido como no original.
Seção 7.1	A mensagem do exemplo de falha (<code>Banco de dados indisponivel.</code>) está sem acento no original; preservada.

Ajustes editoriais aplicados na geração do PDF

- `Controle[ideogramas chineses]` → `Controle granular` (1 ocorrência)
- `SessãoGov.br` → `Sessão Gov.br` (30 ocorrências)
- `autenticacaoGov.br` → `autenticação Gov.br` (1 ocorrência)
- `pode suspende/revogar` → `pode suspender/revogar` (1 ocorrência)

Os ajustes se limitam a caracteres ilegíveis (ideogramas chineses fora de contexto), espaços ausentes e uma palavra truncada; nenhum endpoint, rota, parâmetro, código HTTP ou exemplo foi alterado.